Pre-Viz Engine v1.9.0 QA Report

URL	https://previz-engine-m1mm9ayva-valid.vercel.app/
Date	March 2026
Tester	Bernardo Bugmann (with Claude Code assistance)

---

1. Executive Summary

Pre-Viz Engine is a video pre-visualization tool for filmmakers, powered by OpenAI's Sora API. Users create AI-generated video clips from text prompts through two workflows: direct prompt generation (Studio) and screenplay-based shot extraction (Stock Footage). Generated videos are stored in Supabase and browsable in a history view.

Testing covered the full generation pipeline, from prompt submission through API response, as well as script-to-shot AI parsing, video history browsing, and the API security surface. The automation suite includes 84 Playwright tests across UI, API, accessibility, security, and visual regression layers, running across 4 browser projects (Chromium, Firefox, WebKit, Mobile Chrome) for a total of 310 test executions per run. Known bugs are tracked with `test.fail()` annotations that flip when the underlying issue is resolved.

Release Recommendation: NO-GO

v1.9.0 is not ready for production release. The following blockers must be resolved:

Priority	Bug	Risk
P0	BUG-001: False positive moderation	Innocent prompts like “test” are blocked, so users cannot trust the core generation feature
P0	BUG-002: No rate limiting on `/api/generate-video`	Anyone with the URL can burn unlimited Sora API credits with no throttling
P1	BUG-003: Public Supabase storage bucket	All 2,667+ generated videos are world-readable without authentication
P1	BUG-004: Internal error structure leaked via API	`/api/video-status` exposes raw Sora error messages, revealing backend architecture
P1	BUG-009: Mobile viewport completely broken	4 dedicated mobile tests fail because core workflows are unusable at Pixel 5 resolution

Minimum bar for release: Fix P0 items (moderation accuracy + rate limiting). P1 items are strongly recommended before any public-facing deployment.

---

2. Scope & Environment

Area	Coverage	Method
Studio	Prompt form, generation settings, generation trigger, rendering state	UI tests + mocked API
Stock Footage	Script parsing, shot extraction, editable prompts, edge cases (unicode, malformed, dialogue-heavy)	UI tests + real AI calls
History	Tab filtering, video cards, status badges, playback, pagination	UI tests + API
API Pipeline	generate-video, video-status, video-history endpoints, negative paths	Playwright request context
Security	Rate limiting, error exposure, data integrity, storage access, CSP/HSTS headers, cookie flags	API + automated audits
Prompt Injection	SQL injection, XSS (script/img), path traversal, template literals, unicode/emoji	API + frontend rendering
Accessibility	WCAG 2.1 AA automated scans on all 3 pages	axe-core
Visual Regression	Screenshot baselines for Studio and Stock Footage pages	toHaveScreenshot()

Browser / Device	Viewport	Status
Chromium Desktop	1280x800	Passing
Firefox Desktop	1280x800	Passing
WebKit (Safari) Desktop	1280x800	Passing
Chrome Mobile (Pixel 5)	393x851	9 failures (BUG-009)

Not Tested

Area	Reason
Full E2E generation completion	Each video takes ~5 min via Sora; tests verify trigger + API contracts instead
Video output quality	Subjective, no visual regression baseline for generated content
Accessibility (screen reader testing)	Automated axe-core WCAG 2.1 AA scans added; manual screen reader testing not performed (time constraint)
Load / performance testing	No k6 or Artillery applied to the generation API
Real mobile devices	Time constraint; mobile tested via Playwright device emulation only

---

3. Exploratory Test Charters

Charter 1: Video Generation Pipeline (Studio)

Mission: Verify the complete prompt-to-video flow through the Studio page.

The prompt form correctly validates empty input by disabling the Generate button. Video count (1 to 5), duration (4s, 8s, 12s), and aspect ratio (Landscape, Portrait) controls all work as expected.

When a user triggers generation, the app sends 3 individual POST requests to `/api/generate-video` (one per video variant), each returning a `taskId`. The UI transitions to a “Rendering Pre-Viz Videos” state with progress display and task ID tracking. Status polling via `/api/video-status?taskId=` begins immediately.

The HQ Mode toggle and warning text render correctly, though the warning has a state persistence bug where it persists after page reload even when the toggle resets to off (BUG-007).

The API accepts any prompt text without client-side length validation. Tested with 10,000+ character prompts and all were accepted without warning.

Charter 2: Script-to-Shot Parsing (Stock Footage)

Mission: Test the AI-powered screenplay parsing workflow.

The Parse Script button correctly gates on textarea content, staying disabled when empty. Script parsing calls an AI service with a ~10s response time that extracts individual shots from screenplay format.

A 2-scene screenplay (`INT. COFFEE SHOP / EXT. CITY STREET`) consistently extracts 2+ shots with descriptive prompts. Each extracted shot has an editable textbox pre-filled with an AI-generated visual description, and shot headings show correct numbering (“Shot 1 of N”).

The Start Over button properly resets to the input state, and the Generate button reflects the correct total video count from extracted shots. The default aspect ratio is Portrait (vs. Studio’s Landscape default), which is intentional for the script/storyboard workflow.

Charter 3: History & Playback

Mission: Verify video browsing, status display, and playback functionality.

The History page loads with a “Video History” heading and two tabs: “Your Videos” and “All Videos”. The “Your Videos” tab filters by `previz_user_id` cookie, and a fresh browser correctly shows an empty state message.

The “All Videos” tab shows videos with status badges (Completed, Failed, Pending). Completed videos have playable `<video>` elements, while failed videos display the error message “Your request was blocked by our moderation system.” The “Load More” pagination button works correctly.

Video card metadata (aspect ratio, duration, quality) displays correctly for completed videos.

Charter 4: API Security Surface

Mission: Probe the API for security issues, data integrity, and operational safety.

The `POST /api/generate-video` endpoint has no rate limiting. There are no `x-ratelimit-*` headers and no throttling. Anyone with the URL can trigger unlimited Sora API calls, burning credits (BUG-002).

Querying `GET /api/video-status` with an invalid `taskId` returns HTTP 500 with the raw error message `"Failed to query video generation task"`, exposing internal Sora API error structure (BUG-004).

Video storage in Supabase is world-readable at `bjzbiwehrsaogmqzcpce.supabase.co/storage/v1/object/public/previz-videos/`, with all 2,667+ videos accessible via direct URL (BUG-003).

The Mixpanel analytics token `fc57a921c6f70add4e091ea3d56e1719` is exposed client-side (BUG-006). There is no authentication layer; the app uses only a cookie-based `previz_user_id` for tracking, not authorization. Client-side prompt length is also unbounded (BUG-005).

Charter 5: Cross-Cutting Concerns

Mission: Check responsive layout, error handling, state management, and polish.

Navigation works correctly across all 3 pages with consistent nav links, and the logo link returns to Studio from any page. The 404 page renders for unknown routes with the correct HTTP 404 status code.

There is a missing favicon (404 in network tab, BUG-008). No console errors were observed during normal navigation flows.

Automated axe-core WCAG 2.1 AA scans flagged violations on all 3 pages, including missing accessible names on buttons and select elements, and insufficient color contrast. These are tracked as known failures in the a11y test suite.

Security header audits revealed missing Content-Security-Policy and X-Frame-Options headers. The `previz_user_id` cookie lacks the `HttpOnly` flag. Prompt injection testing (SQL injection, XSS payloads, path traversal) confirmed the API accepts these strings without server errors, and React's default escaping prevents XSS execution in the frontend.

The mobile viewport (Pixel 5, 393x851) has significant issues. Navigation headings are clipped or hidden, the Studio generation form is not fully accessible, Stock Footage script parsing is unusable because form elements are off-screen, and the History "Load More" button is not reachable. In total, 9 tests fail exclusively on mobile-chrome while passing on all desktop browsers (BUG-009).

---

4. Bug Pack

BUG-001: False Positive Moderation on Innocuous Prompts

Severity	Critical
Category	Core Flow

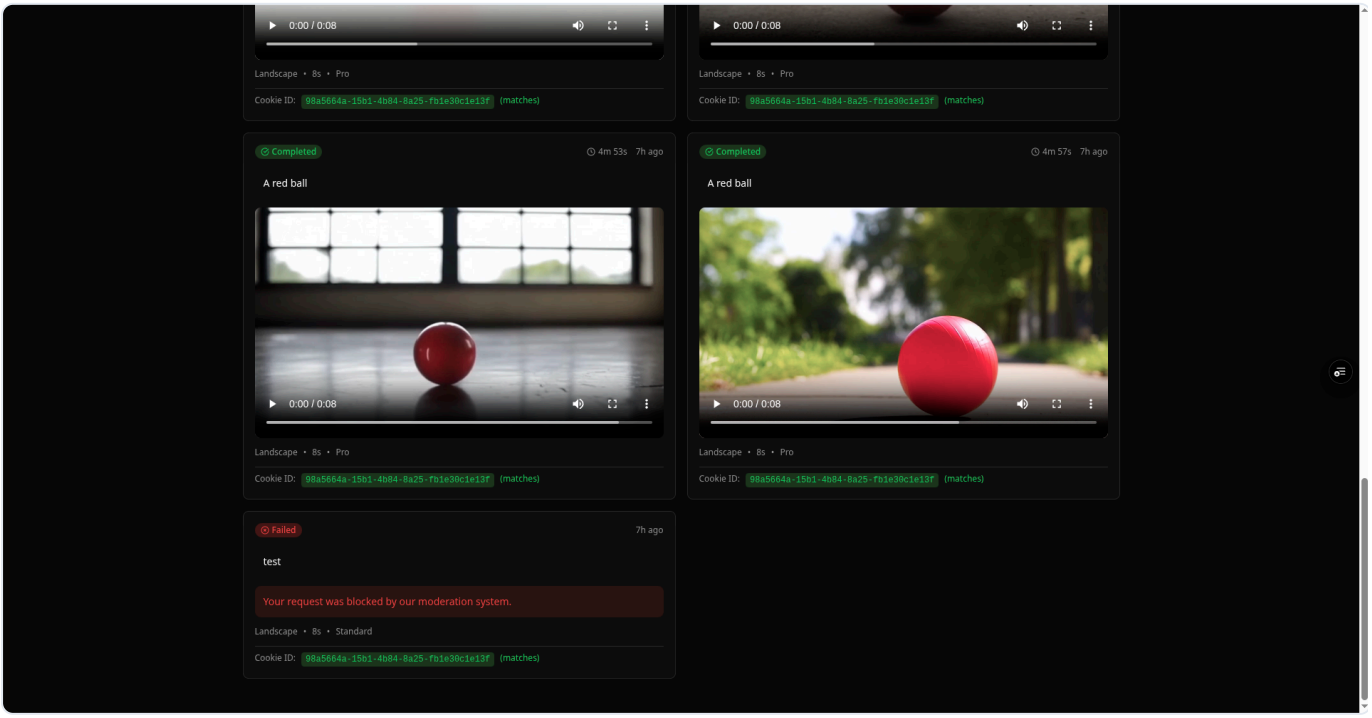
The Sora content moderation system produces false positives on short, innocuous prompts. Videos generated with simple text like "test" are flagged with `content_policy_violation` during processing, resulting in failed generations with the message "Your request was blocked by our moderation system."

Repro Steps: 1. Navigate to Studio page 2. Enter "test" as the prompt 3. Click Generate 4. Wait for generation to complete (~5 min) 5. Check History. The video shows as Failed with moderation error

Expected: Short innocuous prompts should generate successfully. **Actual:** Generation fails with content policy violation.

Users attempting quick tests or short descriptions get blocked, which undermines trust in the core generation feature.

Screenshot:



Observability: Log every moderation decision with the original prompt text, Sora's `content_policy_violation` reason, and a confidence score if available. This enables auditing false positive rates and tuning the moderation threshold.

Automation: `generation-pipeline.spec.ts` "innocent short prompts should not be flagged by moderation" (`test.fail`)

BUG-002: No Rate Limiting on Generation API

Severity	High
Category	Security / Cost

The `POST /api/generate-video` endpoint has no rate limiting whatsoever. No `x-ratelimit-limit` , `x-ratelimit-remaining` , or `x-ratelimit-reset` headers are present. An attacker can trigger unlimited Sora API calls, each consuming OpenAI credits.

Repro Steps: 1. Send `POST /api/generate-video` with a valid payload 2. Inspect response headers. No rate limit headers are present 3. Send 100 rapid requests. All are accepted with 200 status

Expected: Rate limiting headers should be present and requests should be throttled per IP/cookie. **Actual:** No rate limiting exists and unlimited generation requests are accepted.

This is a direct financial exposure since Sora API credits can be burned by anyone with the endpoint URL.

Observability: Add rate limit response headers (`x-ratelimit-limit` , `x-ratelimit-remaining` , `x-ratelimit-reset`). Log every generation request with IP, cookie ID, and timestamp. Set up alerts for anomalous request volume per IP (e.g., >10 requests/minute).

Automation: `generation-pipeline.spec.ts` “generate-video should include rate limiting headers” (`test.fail`)

BUG-003: Public Supabase Storage Bucket

Severity	High
Category	Security

All generated videos are stored in a public Supabase bucket at `bjzbiwehrsaogmqzcpce.supabase.co/storage/v1/object/public/previz-videos/` . The bucket listing is accessible without authentication, exposing all 2,667+ videos to anyone with the URL.

Repro Steps: 1. Open any completed video from History 2. Inspect the `<video>` element’s `src` attribute 3. Navigate to the Supabase storage root. All files are enumerable

Expected: Videos should be access-controlled or served through an authenticated proxy. **Actual:** The entire video library is world-readable.

Observability: Enable Supabase storage access logs. Monitor for bulk enumeration or scraping patterns (sequential object listing, high request volume from single IP). Consider moving to signed URLs with short TTL for video access.

BUG-004: Video-Status API Leaks Internal Error Structure

Severity	Medium
Category	Information Leak

When querying `/api/video-status` with an invalid `taskId` , the API returns HTTP 500 with the raw error message `"Failed to query video generation task"` from the underlying Sora API integration. This exposes internal service architecture.

Repro Steps: 1. `GET /api/video-status?taskId=invalid-task-123` 2. Response: `500 { "error": "Failed to query video generation task: ..." }`

Expected: HTTP 400 with a generic “Invalid task ID” message. **Actual:** HTTP 500 with internal error structure leaked to the client, including Sora API endpoint paths and error format.

Observability: Wrap all Sora API calls in a sanitizing error handler that logs the full error server-side but returns only a generic message to the client. Log invalid `taskId` requests with the caller's IP and cookie to detect enumeration attempts.

Automation: `generation-pipeline.spec.ts` "video-status should return sanitized error for invalid taskId" (`test.fail`)

BUG-005: No Client-Side Prompt Length Validation

Severity	Medium
Category	Input Validation

The prompt textarea accepts unlimited text input. Prompts of 10,000+ characters are sent to the API without any client-side validation or truncation warning.

Repro Steps: 1. Navigate to Studio 2. Paste 10,000+ characters into the prompt textarea 3. Click Generate. The request is sent to the API with the full text

Expected: Client-side validation should warn or prevent excessively long prompts (Sora's practical limit is ~1000 chars). **Actual:** No length validation exists on the client. Any length is accepted and sent to the API.

Observability: Log prompt lengths on the server side. Add a client-side character counter with a warning threshold. Track the distribution of prompt lengths to inform appropriate limits.

BUG-006: Mixpanel Token Exposed Client-Side

Severity	Low
Category	Information Leak

The Mixpanel analytics token `fc57a921c6f70add4e091ea3d56e1719` is visible in the client-side JavaScript bundle and console logs. While Mixpanel tokens are designed to be client-side, the token combined with console logging (`"Mixpanel initialized in lib/mixpanel.ts"`) reveals analytics implementation details.

Repro Steps: 1. Open browser DevTools Console 2. Navigate to any page 3. Observe `"Mixpanel initialized in lib/mixpanel.ts"` log with token reference

Expected: Analytics initialization should not log debug messages or token references in production. **Actual:** Console logs reveal the Mixpanel library file path and initialization details.

Observability: Gate debug logging behind a `NODE_ENV !== 'production'` check. Use environment-specific Mixpanel project tokens so a leaked production token can be rotated without affecting development.

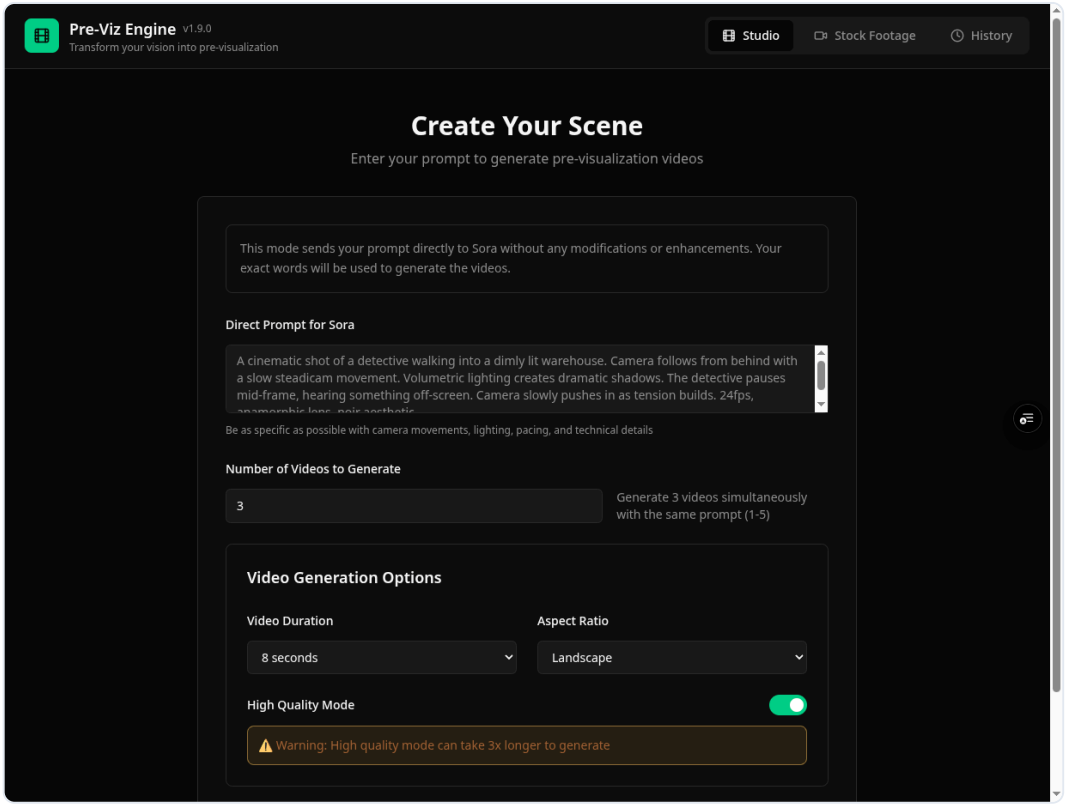
BUG-007: HQ Warning Persists After Page Reload

Severity	Low
Category	UI State

After toggling High Quality Mode on, the warning text (“Warning: High quality mode can take 3x longer to generate”) persists after page reload even though the toggle visually resets to the off position. The warning state and toggle state are out of sync.

Repro Steps: 1. Navigate to Studio 2. Toggle HQ Mode on. The warning appears 3. Reload the page 4. Toggle appears off but warning text is still visible

Expected: Warning should be hidden when HQ toggle is off. **Actual:** Warning persists across page reload regardless of toggle state.



Screenshot:

Observability: The warning visibility should be derived from the toggle state, not stored independently. Check if the warning state is persisted in localStorage or a React state that survives hydration.

Automation: `studio.spec.ts` “HQ warning should be hidden when toggle is off after reload” (`test.fail`)

BUG-008: Missing Favicon and Generic 404 Page

Severity	Low
Category	Polish

The application has no favicon (returns 404 in network tab). The 404 page for unknown routes displays a generic Next.js error page with no Pre-Viz Engine branding.

Expected: A branded favicon should load successfully. The 404 page should match the app's visual identity. **Actual:** Favicon returns HTTP 404. The 404 page shows the default Next.js error page with no Pre-Viz branding.

BUG-009: Mobile Viewport Breaks Multiple Flows

Severity	High
Category	Responsive Layout

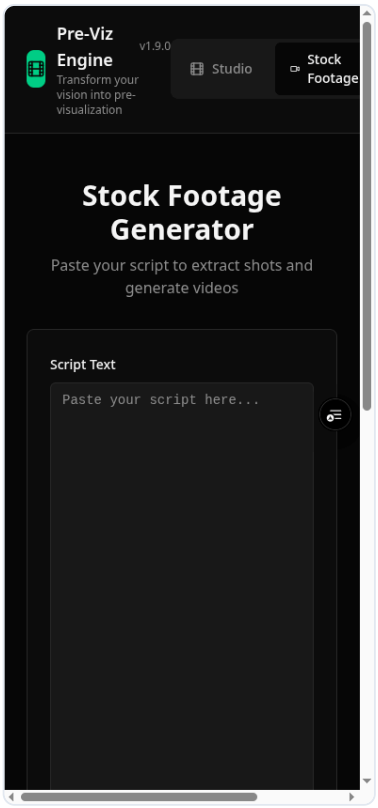
The application is not usable at mobile viewport sizes. At the Pixel 5 resolution (393x851), 4 dedicated mobile tests fail across Studio, Stock Footage, History, and Navigation. Key interactive elements are clipped, hidden, or pushed off-screen, making core workflows impossible to complete on mobile.

Affected flows: - Navigation page headings are truncated or hidden at mobile width - Studio generation form controls (prompt, settings, Generate button) are not fully accessible - Stock Footage script parsing is entirely unusable because form elements are positioned off-screen - History "Load More" pagination button is not reachable

Repro Steps: 1. Open any page using a Pixel 5 viewport (393x851) or similar mobile device 2. Try to complete any generation or script parsing workflow 3. Observe that key form controls and buttons are clipped or unreachable

Expected: All core workflows should be functional at common mobile viewport sizes. **Actual:** 4 dedicated mobile regression tests fail on mobile-chrome while passing on all 3 desktop browsers.

All desktop browsers (Chromium, Firefox, WebKit) pass every test at 1280x800, confirming the issue is strictly viewport-related. This suggests the layout was designed primarily for desktop and lacks responsive breakpoints for mobile.



Screenshot:

Automation: `mobile.spec.ts` has 4 dedicated mobile-only tests with `test.fail` that document each broken flow.

BUG-010: API Accepts Arbitrarily Large Limit Values

Severity	Medium
Category	Input Validation

The `GET /api/video-history` endpoint accepts any `limit` value without server-side capping. Requesting `?limit=99999` returns all ~1000+ videos in a single response instead of enforcing a reasonable maximum (e.g., 100).

Repro Steps: 1. `GET /api/video-history?filter=all&limit=99999` 2. Response: 200 with 1000+ video objects in a single payload

Expected: Server should cap the limit parameter to a reasonable maximum (e.g., 100) regardless of the requested value. **Actual:** The API returns up to 1000 videos in a single response, creating large payloads that could be used for scraping or denial-of-service.

Observability: Add server-side limit capping with a maximum of 100. Log requests with unusually high limit values as potential abuse indicators.

Automation: `negative-paths.spec.ts` “video-history with huge limit is handled” (`test.fail`)

---

5. How I Used Claude Code

The Workflow

My approach combined Claude Code with two plugins: the Playwright MCP browser plugin for live browser control, and Superpowers for structured workflows like code review, plan execution, and parallel agent dispatch. Together they created a tight feedback loop where I could explore the app live, identify behaviors, write tests, review them against requirements, and verify everything against the real application without switching tools.

The entire process happened inside the terminal. Claude Code controlled a real browser instance through the Playwright plugin, which let me observe DOM state, intercept network requests, evaluate JavaScript in the page context, and capture accessibility snapshots. Superpowers enforced discipline: brainstorming before implementation, code review after each major step, and verification before claiming anything was done. This was significantly faster than alt-tabbing between a browser, editor, and task tracker.

Exploration Phase

Prompt 1: Mapping the application

> *“Navigate to the Pre-Viz Engine URL, take snapshots of all pages, and map the app structure: pages, navigation, API calls, and key UI elements.”*

Claude Code visited each page (`/` , `/stock-footage` , `/history`) and returned accessibility snapshots showing every heading, button, form control, and link. Console logs revealed Mixpanel initialization with the token in plain text. The History page snapshot immediately exposed the Cookie Debug Info panel. This single prompt gave me a complete mental model of the app in under a minute.

Prompt 2: Probing the API surface

> *“Use page.evaluate to call each API endpoint directly. Test generate-video with valid and invalid payloads, video-status with real and fake taskIds, and video-history with different filters. Check response headers for rate limiting.”*

This was the most productive prompt. Claude Code ran `fetch()` calls inside the browser and reported back exact response shapes, status codes, and headers. The missing rate limit headers showed up immediately. The invalid taskId returned a 500 with raw Sora error internals instead of a clean 400. I followed this up by

asking Claude to inspect the `<video>` element sources, which led to discovering the world-readable Supabase bucket.

Prompt 3: Triggering real video generation

> *"Fill in a prompt on the Studio page, click Generate, and watch the network requests. I need to know the exact API request/response shapes for mocking."*

This was where things got interesting. I expected a single batch POST, but Claude Code observed 3 separate POST requests to `/api/generate-video`, one for each video variant. Each returned `{ taskId, duration_ms, prompt_length, aspect_ratio, seconds, pro }`. Without this observation, the mock responses I wrote later would have been completely wrong. In fact, the first mock attempt used a batch shape (`{ tasks: [...] }`), and the app showed "Video Generation Failed" until I corrected it to match the real individual-response pattern.

Test Development Phase

Prompt 4: Initial test suite

> *"Write Playwright tests covering navigation, studio form controls, and the generation trigger flow. Use the API response shapes we discovered for mocking."*

Claude Code produced a working test suite, but the first version had several issues I caught in review:

The tests were organized by test type (`smoke/`, `regression/`) instead of by business feature. I pushed back and asked for a reorganization into `ui/` and `api/` folders, which better reflects how the application works and how a team would actually run subsets of tests.

There were no Page Object Models. Locators were scattered across spec files, making them fragile and hard to maintain. I asked for POMs for each page, Playwright custom fixtures to inject them automatically, and a rule: locators belong in page files, assertions belong in spec files.

There was no ESLint. I asked for `eslint-plugin-playwright` with flat config, which immediately caught anti-patterns like conditional expects and legacy selectors.

Prompt 5: Script parsing tests

> *"Navigate to Stock Footage, paste a 2-scene screenplay, click Parse Script, and observe the full parsing flow. Then write tests for it."*

Claude Code observed the real AI parsing call (~10s) and the resulting DOM changes: shot headings with numbering, editable textboxes with AI-generated descriptions, and a Generate button reflecting the total shot count. The tests for this section use a 60s timeout since they hit the real AI endpoint. I considered mocking it, but the parsing quality is the whole point of this feature, so real calls made more sense.

Prompt 6: Security surface deep dive

> *"Audit the security headers on all responses. Check for CSP, HSTS, X-Frame-Options, X-Content-Type-Options. Then check the previz_user_id cookie for Secure, HttpOnly, and SameSite flags. Also test prompt injection: send SQL injection, XSS payloads, path traversal, and template literals as prompts and verify the app doesn't crash or leak data."*

This prompt drove the security hardening phase. Claude Code wrote the security-headers audit suite and the prompt edge cases suite in one pass. The interesting finding was that XSS payloads in prompts are stored verbatim in the API response JSON, but React's default escaping prevents execution in the frontend. I had to correct the agent's first XSS test because it was checking whether `<script>alert` appeared in the API JSON response, which is the wrong layer. The real test is whether the frontend renders it as executable HTML, which it doesn't.

Prompt 7: Accessibility and visual regression

> *"Add axe-core WCAG 2.1 AA scans for each page. Attach the violation details as JSON artifacts in the test report, not console.log. Also set up visual regression baselines with toHaveScreenshot for Studio and Stock Footage."*

Claude Code initially used `console.log` to dump accessibility violations, which is useless in a CI report. I pushed back and asked for `test.info().attach()` with JSON artifacts so violations appear directly in the Playwright HTML report. The a11y scans revealed real WCAG violations on all 3 pages (missing button names, poor color contrast), which are now tracked as known failures.

What the Agent Got Wrong

The biggest issue was a generic web app testing mindset. The first pass focused on navigation links, form validation, and basic page loads. This is a video generation tool powered by an AI API. The testing should center on the generation pipeline, the AI parsing quality, and the security surface, not on whether three nav links are visible.

Initial bug findings were also too shallow: missing favicon, nav label truncation at small viewports. I had to redirect toward the API security surface, which uncovered the serious issues like missing rate limiting, public storage buckets, and internal error structure leaks.

The mock API response shape was wrong on the first try. The agent assumed the API would use a batch pattern (`{ tasks: [...] }`) when it actually fires individual requests per video. This is a good example of why you need to observe real behavior before writing mocks.

Test organization needed a full refactor. Moving from type-based folders to feature-based folders, adding POMs, fixtures, barrel files, and tag-based scripts was the right call but should have been the starting point, not a retroactive fix.

How I Verified Correctness

Every test assertion was validated against real application behavior observed through the MCP browser plugin. API response shapes came from actual `fetch()` calls evaluated in the browser context, not from documentation or assumptions.

I ran the full automation suite against the live app after each batch of changes and cross-referenced results with what I could see in the browser snapshots. When something failed, the MCP plugin let me immediately inspect the page state to understand why.

Bug regression tests use `test.fail()` with the correct expected behavior as the assertion. This means when a bug is fixed, the test unexpectedly passes, which triggers a failure alerting the team to remove the `test.fail` annotation. This is a deliberate design choice that keeps bugs tracked inside the test suite instead of in a separate document.

What I'd Automate Next

Full E2E generation with mocked Sora backend. Right now tests verify the trigger and the API contracts separately. A mocked backend returning instant "completed" responses would let us test the entire pipeline, from prompt to video playback, in seconds instead of 5 minutes per video.

Moderation accuracy matrix. A dedicated suite maintaining a list of innocuous prompts that should pass and harmful prompts that should fail, tracking false positive and false negative rates over time. This turns BUG-001 from a single observation into a monitored metric.

Load testing with k6. The missing rate limiting (BUG-002) needs quantification. How many concurrent generations can the API handle before degrading? What's the actual per-minute cost exposure? k6 scripts targeting `/api/generate-video` with ramping virtual users would answer this.

Contract testing for Sora API changes. The mock response shapes in `studio.page.ts` are based on observed real API behavior. If Sora's response format changes, tests would still pass against the stale mocks while the real app breaks. A lightweight contract test hitting the real API periodically would catch drift.